2025 uni-app面试题汇总

1.uniapp调用支付

- 步骤
  a. 创建订单
    - 组织订单的信息对象
    - 发起请求创建订单
    - 得到服务器响应的“订单编号”
  b. 订单预支付
    - 发起请求获取订单的支付信息
    - 预付订单生成失败 **提示**
    - 得到订单支付相关的必要参数

```
let timeStamp = payInfo.timeStamp
let nonceStr = payInfo.nonceStr
let packageX = payInfo.package
let signType = payInfo.signType
let paySign = payInfo.paySign
```

  c. 发起微信支付
    - 调用 uni.requestPayment() 发起微信支付

```
uni.requestPayment({
    provider: 'wxpay',  //
    package: packageX,
    timeStamp: timeStamp,
    nonceStr: nonceStr,
    signType: signType,
    paySign: paySign,
    success(res) {
        uni.$showMessage(res)
    },
    fail(err) {
        uni.$showMessage(err.errMsg)
    }
```

```
})
```

- 未完成支付 **提示**
- 完成了支付，进一步查询支付的结果
- 检测到订单未支付 **提示**
- 检测到订单支付完成 **提示**

2.微信小程序知识点

- 微信小程序的主要文件
  - WXML——模板文件
  - JSON——配置/设置文件，如标题，tabbar，页面注册
  - WXSS——样式文件，样式可直接用 import 导入
  - JS——脚本逻辑文件，逻辑处理，网络请求
  - app.json——配置文件入口，整个小程序的全局配置
  - app.js——可以没有内容，可以在里边监听生命周期函数、声明全局变 量
  - app.wxss——全局配置样式文件
- 传值方法
  - navigator 组件或 wx.navigator
  - 本地存储
  - app.js 中的全局对象
  - 自定义属性 data-xxx
- 登录流程
  - 通过wx.login()获取用户的code判断用户是否授权读取用户信息——>调用wx.getUserInfo()自身后端调用微信服务器读取用户数据——>通过 wx.request()请求业务方服务器端把 appid，appsecret 和 code 一起发送到微信服务器——>微信服务器返回了 openid 及本次登录的会话密钥 session_key——>然后生成 session 并返回给小程序
- 小程序内页面跳转
  - wx.navigateTo——>**保留当前页面，跳转到应用内的某个非tabbar页面**（参数必须为字符串）
  - wx.redirectTo——>**关闭当前页面，跳转到应用内的某个非tabbar页面**
  - wx.switchTab——>**跳转到 tabBar 页面，并关闭其他所有非 tabBar 页面， 路径后不能带参数**
  - wx.navigateBack——>关闭当前页面，返回上一页面或多级页面
  - wx.reLaunch——>关闭所有页面，打开到应用内的某个页面

- 小程序的生命周期
  - 应用的生命周期

生命周期	说明
onLaunch	小程序初始化完成时触发，全局只触发一次
onShow	小程序启动，或从后台进入前台显示时触发
onHide	小程序从前台进入后台时触发
onError	小程序发生脚本错误或 API 调用报错时触发

生命周期		说明
onShow	生命周期回调—监听页面显示	请求数据
onReady	生命周期回调—监听页面初次渲染完成	获取页面元素（少用）
onHide	生命周期回调—监听页面隐藏	终止任务，如定时器或者 播放音乐
onUnload	生命周期回调—监听页面卸载	终止任务
onLoad	生命周期回调—监听页面加载	发送请求获取数据

- 小程序的组件通信
  - 父传子
    i. 在子组件的组件标签上通过自定义属性的形式绑定数据或字符串
    ii. 在子组件中通过 properties 对象进行属性的接收即可。
  - 子传父
    i. 在子组件中的 methods 对象中定义方法，在方法中通过 this.triggerEvent({})方法，完成事件触发
    ii. 在子组件标签上绑定（例：bind：在 this.triggerEvent 定义的事件名称=" 回调函数"），在 this.triggerEvent 定义的事情名称，最后在回调函数中完成逻辑 处理
- **小程序设置背景图不展示**

- 本地图片转换为base64格式
- 使用行内样式
- image组件替换背景图片，定位实现背景图片效果

3. uni-app跨端原理

uni-app 分 **编译器和运行时（runtime）** ，实现一套代码，多端运行主要是这两部分配合完成的， 编译器将开发者的代码进行编译，编译的输出物由每个平台各自的runtime进行解析。

微信小程序登陆流程

- 通过wx.login()调用小程序api获取登陆凭证code，将code传给服务端
- 服务端调用微信的登陆接口，把appid（小程序ID）,appsecret（小程序密钥）、code码传给微信后台
  - appID的获取（微信小程序后台---开发--开发设置---小程序ID）
  - appsecret的获取（微信小程序后台---开发--开发设置---小程序密钥）
- 微信后台验证成功后，返回给服务器用户的唯一标识（openid）和本次登陆的会话密钥(session_key)
- 服务端会与openid相关联生成token，返回给小程序端
- 小程序端将token存入缓存，再次向服务端发送请求时，携带token，服务端验证token后返回小程序需要的业务数据

4. uni-app的生命周期

应用生命周期

```
onLanuch - uni-app 初始化完成时触发（全局只触发一次）
onShow - uni-app启动，或从后台进入前台显示
onHide - uni-app从前台进入后台
onError - 当uni-app 报错时触发
onUNiNViewMessage - 对 nvue 页面发送的数据进行监听
onUnhandledRejection - 对未处理的Promise拒绝事件监听函数
onPageNotFound - 页面不存在监听函数
onThemeChange - 监听系统主题变化
```

页面生命周期

```
onInit - 监听页面初始化，参数同onLoad参数，为上个页面传递的数据，参数类型为Object，触发时机早于onLoad
onLoad - 监听页面加载，其参数为上个页面传递的数据，参数类型为Object
onShow - 监听页面显示，页面每次出现在屏幕上都触发，包括从下级页面返回露出当前页面
onReady - 监听页面初次渲染
onHide - 监听页面隐藏
onUnload - 监听页面隐藏
onResize - 监听窗口尺寸变化
```

组件生命周期（vue的生命周期)

uniappA页面传值

- 第一种：URL参数传递 :通过在跳转链接中添加参数，在目标页面通过 `this.$route.params` 获取传递的参数

```
<!-- 跳转页面 -->
<uni-link :url="'/pages/targetPage/targetPage?param1=' + value1 + '&param2=' + value2">跳转到目标页面</uni-link>
```

```
// 在目标页面获取参数
export default {
  mounted() {
    const param1 = this.$route.params.param1;
    const param2 = this.$route.params.param2;
    console.log(param1, param2);
  }
}
```

- 第二种：Vuex状态管理
- 第三种：本地存储：使用uni.setStorage和uni.getStorage等方法，将数据存储在本地，在另一个页面读取

```
// 在页面A中保存数据到本地存储
uni.setStorage({
  key: 'yourDataKey',
  data: yourData,
});
```

```
// 在页面B中从本地存储中读取数据
uni.getStorage({
  key: 'yourDataKey',
  success: function (res) {
 const pageData = res.data;
  },
});
```

5. uni-app实现跨端适配

使用条件编译模式，对js代码、css、template在某个环境中生效。 `条件编译 以 #ifdef+环境名开头 以#endif 结尾， 限制一段代码只在某个平台存在`

以 #ifndef+环境名 开头 以#endif 结尾, 限制一段代码除了某平台均存在

```
/* uni.scss*/
/* #ifdef H5*/
.class{
 color:red
}
/* #endif*/
```

6. uniapp封装组件

`u-uploader`： （`同时上传图片和视频`）

背景：官方上传只能上传图片，不能同时上传图片和视频的组件

`全局水印组件`： （信息展示的保密性）

7. uniapp封装方法

- Toast提示

```
/**
 * 提示方法
 * @param {String} title 提示文字
 * @param {String}  icon icon图片
 * @param {Number}  duration 提示时间
 */
export function toast(title, icon = 'none', duration = 1500) {
    if(title) {
        uni.showToast({
```

```
            title,
            icon,
            duration
        })
    }
}
```

- 图片预览

```
/**
 * 预览图片
 * @param {Array} urls 图片链接
 */
export function previewImage(urls, itemList = ['发送给朋友', '保存图片', '收藏']) {
    uni.previewImage({
        urls,
        longPressActions: {
            itemList,
            fail: function (error) {
                console.error(error,'===previewImage')
            }
        }
    })
}
```

- 图片下载

```
/**
 * 保存图片到本地
 * @param {String} filePath 图片临时路径
 **/
export function saveImage(filePath) {
    if (!filePath) return false
    uni.saveImageToPhotosAlbum({
        filePath,
        success: (res) => {
            toast('图片保存成功', 'success')
        },
        fail: (err) => {
            if (err.errMsg === 'saveImageToPhotosAlbum:fail:auth denied' || err.errMsg === 'saveImageToPhotosAlbum:fail auth deny') {
                uni.showModal({
```

```
                    title: '提示',
                    content: '需要您授权保存相册',
                    showCancel: false,
                    success: (modalSuccess) => {
                        uni.openSetting({
                            success(settingdata) {
                                if (settingdata.authSetting['scope.writePhotosAlbum']) {
                                    uni.showModal({
                                        title: '提示',
                                        content: '获取权限成功,再次点击图片即可保存',
                                        showCancel: false
                                    })
                                } else {
                                    uni.showModal({
                                        title: '提示',
                                        content: '获取权限失败，将无法保存到相册哦~',
                                        showCancel: false
                                    })
                                }
                            },
                            fail(failData) {
                                console.log('failData', failData)
                            }
                        })
                    }
                })
            }
        }
    })
}
```

8. Uni-app分包策略

背景

微信小程序之所以需要分包，主要是为了解决小程序官方限制了主包体积和总体积的大小，如果应用体积超限，我们将不能发布到应用官方，最终会上不了线。`整个小程序所有分包大小不能超过20M，单个分包/主包大小不能超过2M`

核心思路：

`将代码划分成不同的包，打开一个包中的某个 页面，才加载这个包的代码。优化小程序首次启动的下载时间。`

- 主包：默认启动页面/TabBar(标签)页面，以及一些所有分包需要用到的公共资源/JS脚本
- 分包：根据开发者的配置进行划分
  a. 使用 `subpackages 进行分包路径声明`，subpackages 配置路径外的目录会被打包到主包中
  b. `tabBar 里配置的路径必须放在主包里`
  c. `不同的分包之间的资源不能相互引用，但都可引用主包中的资源`

TabBar标签页面：

分包的好处:

- `提高首页加载速度`：随着小程序项目规模的增大，首页所需的代码和资源也会越来越多，导致首页加载时间变长，影响用户体验。通过分包，可以将部分代码和资源拆分到其他子包中，在首页加载时只需加载必需的核心代码，从而减少首页的加载时间。
- `优化性能`：小程序的性能对用户体验至关重要。通过分包，可以将一些与首页无关的功能模块或页面、大型资源文件等拆分到子包中，子包的使用也可以帮助有效减少小程序包的体积，提升小程序的加载速度。
- `分包预下载`：分包可以提前加载用户即将使用的功能模块，从而加快跳转到对应页面的速度。通过合理的分包策略和预下载机制，可以在用户交互前就将页面所需的代码和资源提前加载好，确保用户流畅的使用体验。

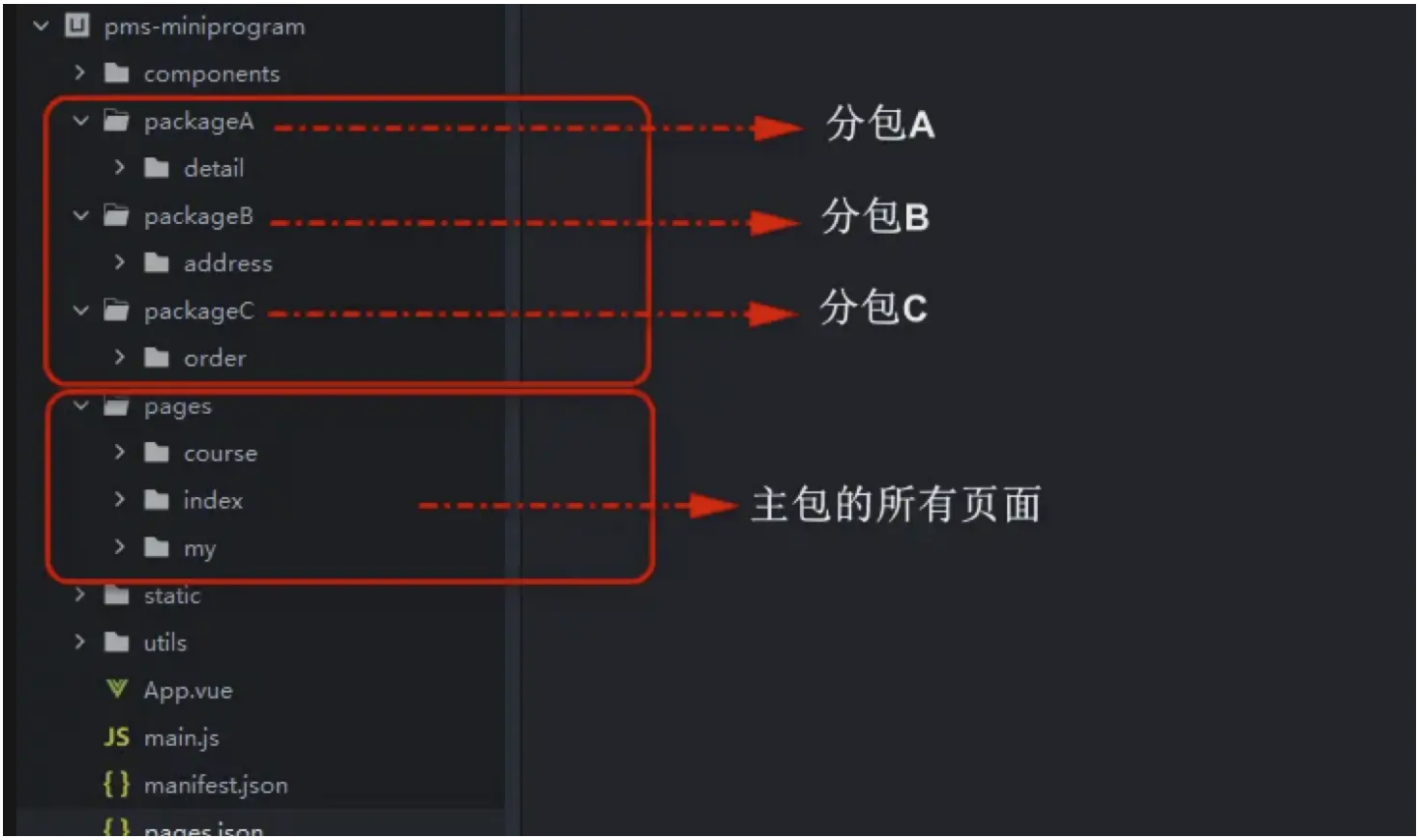
制定合理的分包策略

小程序包拆分为主包和子包，其中主包包含了小程序的首页和一些常用基础功能模块，而子包则包含了其他功能模块和页面。主包在用户第一次打开小程序时会被下载和加载，而子包则根据需要来动态下载和加载。 制定分包策略的建议：

- `根据功能模块拆分`：将小程序的功能模块拆分成不同的子包。比如：tabbar 模块、用户模块、推送模块等等。
- `根据资源引用拆分`：自定义组件、JS 文件、静态资源仅被一个分包使用时则把它划为同一个分包中，如果是公共的资源被各个分包使用，则将其划为主包子云啊
- `分包预下载配置`：通过分包预下载机制，在用户需要时能够快速加载，配置 preloadRule 后，在进入小程序某个页面时，由框架自动预下载可能需要的分包，提升进入后续分包页面时的启动速度，减少用户等待时间，提升用户体验。

具体分包操作

- 项目的目录结构



- 使用 `optimization` 属性开启分包优化
  - 在 `manifest.json` 文件中加入 `"optimization": {"subPackages": true}`
- 在 `pages.json` ，使用 `subPackages` 配置分包信息，定义每个子包的路径、名称和需要包含的页面

```
// pages.json
{
  //主包：只存放Tabbar页面及公共页面
  “pages”:[

  ],
  "subPackages":[
    {
      "root":"packageA",  //分包的根目录
      "pages":[{"path":"detail/index"}]  //该分包下的所有页面
    }
  ]
}
```

分包预下载

分包预下载：进入小程序某个页面时，框架自动预下载可能需要的分包，提升进入后续分包页面时的启动速度。 在pages.json文件的preloadRule节点中配置分包预下载规则，预下载的行为，会在进入指定的页面时触发。

```
{
  "preloadRule": { // 分包预下载规则配置
    "packageA/detail/index": { // 触发分包预下载的页面路径
      // network 表示在指定的网络模式下进行预下载
      // 可选值为：all（不限网络） 和 wifi（仅 wifi 模式下进行预下载）
      // 默认值为：wifi
      "network": "all",
      // packages 表示进入页面后预下载哪些分包
      // 可以通过 root 或 name 指定预下载哪些分包
      // 如果是 __APP__ 表示下载所有包
      "packages": ["packageA"]
    }
  },
}
```

9. 谈谈你对uni-app的理解

uni-app是一个使用Vue.js开发所有前端应用的框架，开发者编写一套代码，可以发布到IOS、Android、Web（响应式）、以及各种小程序、快应用等多个平台。

uniapp真正做到**一套代码多端发行，支持原生代码混写和原生sdk集成**。

运行体验更好。组件、api与微信小程序一致，兼容 Weex 原生渲染。

通用技术栈，学习成本更低。Vue的语法，微信小程序的api，对于前端开发人员来说更容易上手。

开放生态，组件更丰富。支持通过npm安装第三方包；支持微信小程序自定义组件及sdk；兼容 mpvue 组件及项目；app端支持与原生混合编码；

1. uni中如何为不同的平台设置不同的代码

通过**条件注释**来为不同平台设置不同的代码：

条件注释的作用：**实现跨端兼容**

使用方法：以 **#ifdef** 或 **#ifndef** 加平台代值开头，以 **#endif** 结尾

#ifdef：if defined 如果是xx平台则运行代码块 #ifndef：if not defined 如果不是xx平台才运行代码块

- H5　　H5
- MP-WEIXIN　　微信小程序

- APP-PLUS app
- MP 所有小程序

10. uni支持的文件类型

- .vue
- .js
- .css
- pages.json
- 各预编译语言文件，如：.scss、.less、.stylus、.ts、.pug

11. uniapp中封装接口请求相较于微信小程序有什么要注意的

uniapp和微信小程序都提供了网络请求API（uni.request(OBJECT) 和 wx.request(object)），但uniapp 为了实现跨端兼容，需要注意网络请求**跨域问题**，微信小程序不用考虑多端兼容，也不会出现跨域问题。

uniapp中的跨域问题依然可以通过在 **vue.config.js** 中配置**Proxy代理**解决；

devServer ——> proxy ——> changeOrigin: true, //是否跨域

12. uni-app 在非h5端上运行为什么要在架构上分为逻辑层和视图层？

主要原因是**性能**。web端都运行在webview里，js运算和界面渲染会抢资源导致卡顿，而小程序和app，逻辑层都独立为了单独的 js 引擎，渲染层仍然是webvbiew（app也支持原生渲染）。所以在小程序和app上不支持window，dom等API（app可以在渲染层操作window,dom）

13. 详细描述一下Uniapp的工作原理。

将Vue.js的**模板语法**转换为各个平台所支持的**原生组件**，再通过**引擎的渲染器**将其渲染到各个平台上。

同时，Uniapp还提供了平台API以及对各平台的配置文件支持，使得开发者能够方便地在同一份代码下实现对不同平台的适配。

14. 描述一下在Uniapp中如何实现跨平台开发。

在Uniapp中，使用Vue.js的语法开发，同时通过Uniapp所提供的各平台API和配置文件适配各种平台的差异即可实现跨平台开发。例如，在编写页面的时候，可使用Uniapp提供的原生组件替代HTML标签，然后在编写样式时，可通过编写针对各平台的样式代码，使样式在各平台下呈现一致。

15. 在Uniapp中是否可以使用原生功能？如果可以，如何实现？

是的，Uniapp支持使用原生功能。具体实现方式是，可以使用Uniapp提供的API调用各平台的原生组件和接口，然后再将其渲染到对应的平台上。比如，在调用原生摄像头时，可以通过调用Uniapp提供的API获取到原生摄像头组件，并在页面中进行渲染。

16. 在Uniapp中如何处理网络请求?

在Uniapp中，可通过Uniapp提供的网络请求方法来处理网络请求。具体实现方式是，在前端代码中编写网络请求代码，使用Uniapp提供的request方法发送请求，并在回调函数中处理响应结果。

17. 描述一下在Uniapp中，组件和页面的区别。

在Uniapp中，页面和组件的区别在于，页面是一个具体的应用页面，拥有独立的路由地址和生命周期函数，而组件是应用页面中的局部组件，包含在页面中，没有独立的路由地址和生命周期函数。

18. Uniapp如何实现自定义组件?

在Uniapp中，可通过Vue.js的组件机制实现自定义组件。具体实现方式是，编写组件的基本结构和属性方法等，并将组件注册到Vue.js的全局组件或局部组件中，然后在需要使用组件的地方进行调用。

19. 请列出Uniapp工程中有哪些可用的构建模式?

Uniapp工程中可用的构建模式有：**开发模式、生产模式、H5模式、跨平台模式**等。

20. 描述一下Uniapp的几种布局方式。

在Uniapp中，可实现的布局方式有**Flex布局、Grid布局、绝对布局**等。这些布局方式可根据不同的场景来选择使用。

21. 如何在Uniapp中使用vuex来管理全局状态?

在Uniapp中，可通过vuex来管理全局状态。具体实现方式是，在应用程序的入口文件中，注入vuex实例，并在其中编写状态管理器，然后在各组件中使用mapState、mapGetters、mapMutations等方法来访问和修改全局状态。

22. 在Uniapp中，如何使用原生SDK以及插件?

在Uniapp中，可通过uni-app plus的方式来集成原生SDK和插件。具体实现方式是，在**manifest.json**文件中添加对应的插件设置，并在代码中调用相关API使用内置的SDK和插件。

23. 描述一下在Uniapp中如何实现动态路由。

在Uniapp中，可通过vue-router实现动态路由。具体实现方式是，在定义路由时，使用动态路由参数设置路由的路径，然后在组件中通过$route.params来获取路由参数，以便进行页面的动态渲染。

24. 一句话总的形容uniapp与vue和微信小程序的区别

uni-app就是用着vue的指令和小程序的组件和API

25. uni-app 跨端原理

uni-app 分编译器和运行时（runtime），实现一套代码，多端运行主要是这两部分配合完成的 编译器将开发者的代码进行编译，编译的输出物由每个平台各自的runtime进行解析。\

26. 不同平台的 runtime 是怎么转义的?

小程序端，使用小程序版的vue runtime ，页面路由，组件，api 等方面基本都是转义。 web 端，uni-app的runtime 相比普通的vue项目，多一套ui库，页面路由框架，uni对象。 App 端，uni-app的runtime 更复杂，DCloud 有一套小程序引擎，打包app时将开发者的代码和DCloud 的小程序打包成apk或ipa

27. uni-app 的编译器是如何特定编译的?

在 **web**、**ap**p平台，将.vue 文件 编译成 **js 代码**，小程序则拆分shengchengwx,l,wxss ,js等 如果涉及**uts**代码，安卓编译为**kotlin**代码，ios 编译成**swift** 代码。 vue2版本的编译器基于** webpack** 实现，vue3通过 **vite** 实现，性能更快。 同时也支持条件编译，可以指定代码至编译到特定的终端平台。